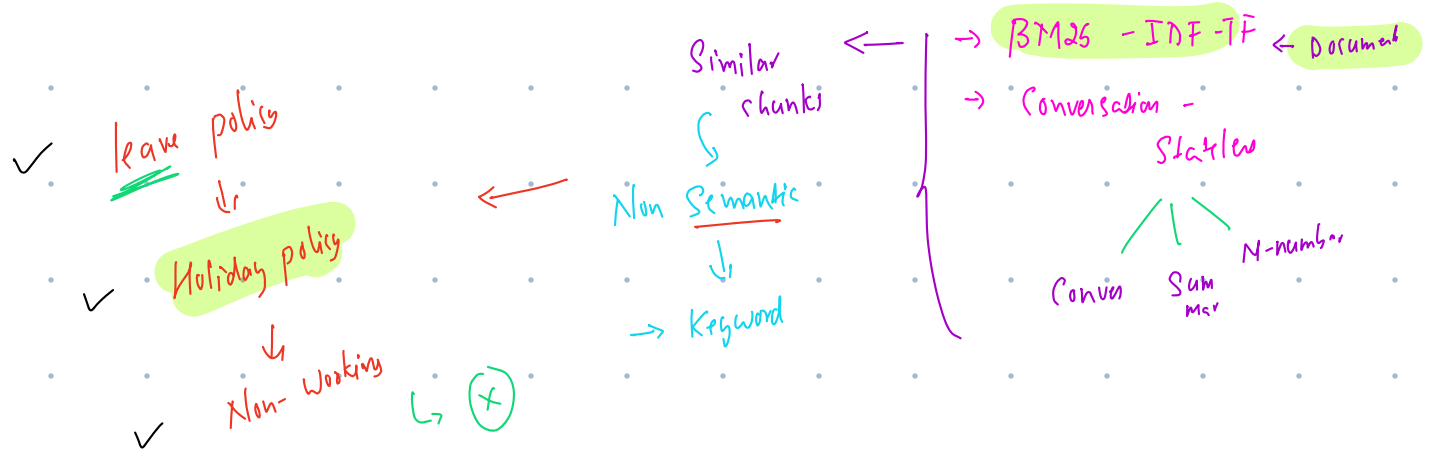
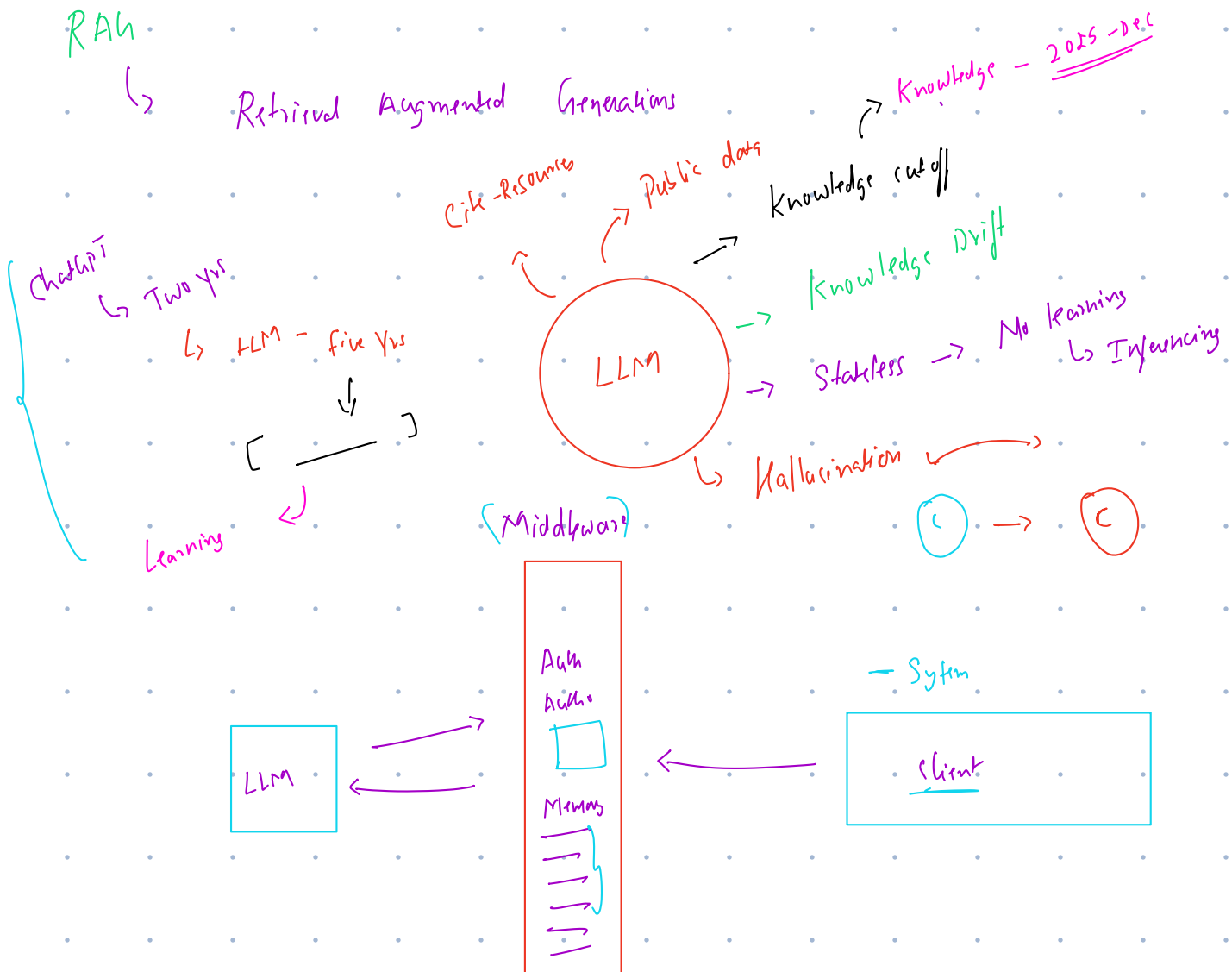


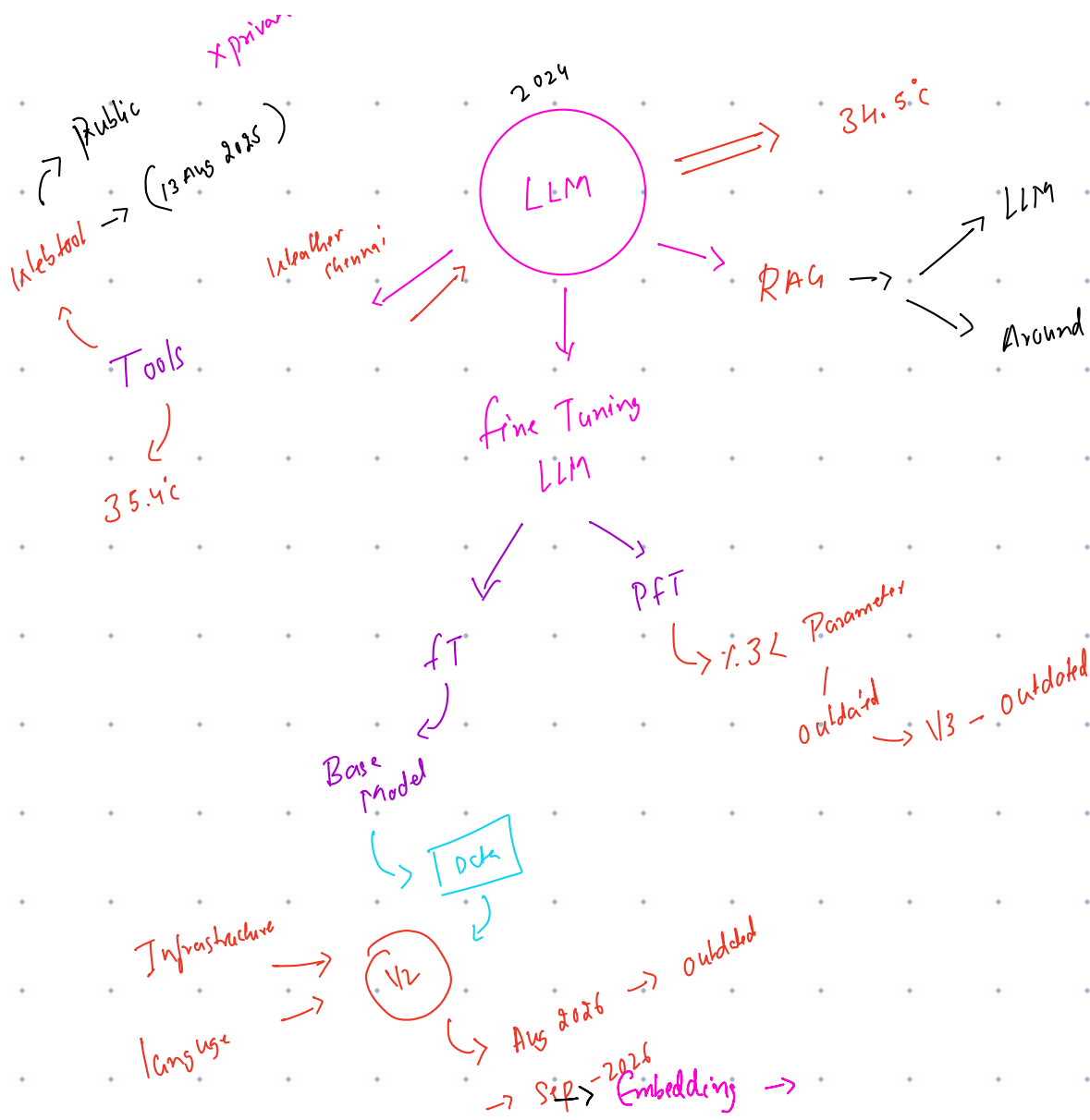


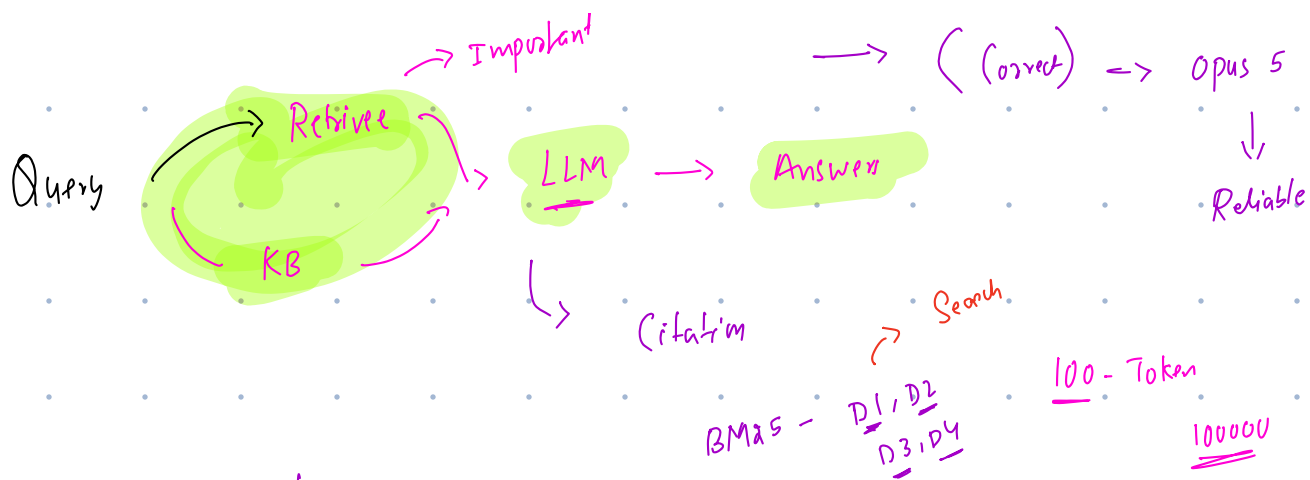
RAH

↳ Retrieval Augmented Generations

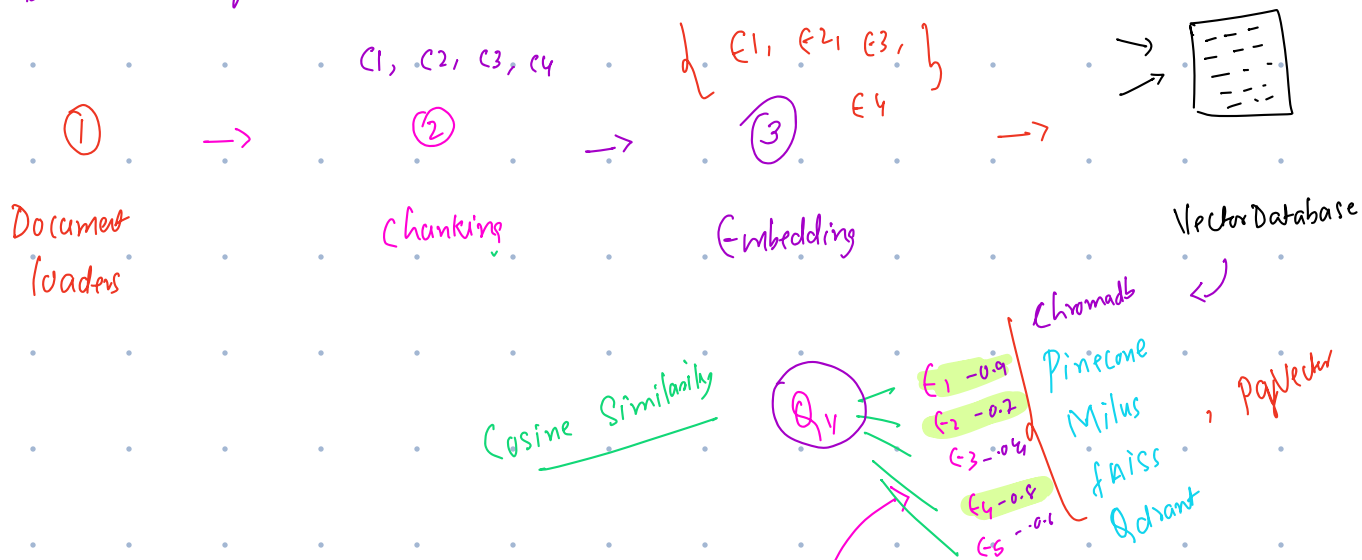


Org → Push → Accountability → Reliability





① Data Ingestion



② Querying/ Retrieval



③ Generation

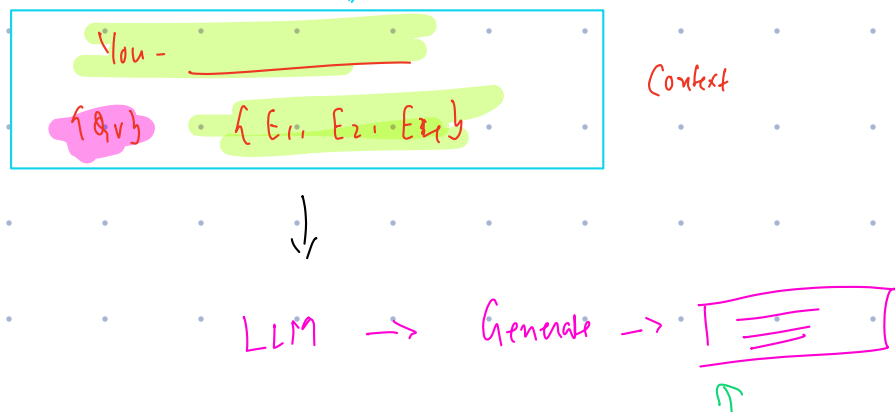


Diagram illustrating the concept of token counts and their relationship to embedding dimensions:

- Columns (Token Counts):**
 - 100 Token
 - 1000 Tokens
 - 10,000 Token
- Context (C) and Embedding (E) Labels:**
 - C1, C2, C3 (Context labels)
 - E1, E2, E3 (Embedding labels)
- Connections:**
 - Curved lines connect C1 to E1, C2 to E2, and C3 to E3.
 - Vertical lines connect E1 to E2, E2 to E3, and E3 to E1.
 - Vertical lines connect E1 to E2, E2 to E3, and E3 to E1.
 - Dashed lines connect E1, E2, and E3 to a central point labeled "5x-Weightage".
 - Vertical lines connect E1 to E2, E2 to E3, and E3 to E1.
- Additional Labels:**
 - Embedding - 512
 - 512
 - 500 Token-Channel

How Much - Embedding Model

Model	Dimensions	Max Tokens	Best For	Cost
text-embedding-3-small (OpenAI)	1536	8191	General use, production	Low (API)
text-embedding-3-large (OpenAI)	3072	8191	High-accuracy tasks	Medium (API)
text-embedding-ada-002 (OpenAI)	1536	8191	Legacy, widely used	Low (API)
all-MiniLM-L6-v2 (Sentence-Transformers)	384	256	Fast, local, low cost	Free (local)
all-mpnet-base-v2 (Sentence-Transformers)	768	512	High quality local	Free (local)
bge-large-en-v1.5 (BAAI)	1024	512	Best open-source quality	Free (local)
e5-large-v2 (Microsoft)	1024	512	Strong retrieval perf.	Free (local)
Cohere embed-english-v3.0	1024	512	Reranking-friendly	Low (API)
Voyage AI voyage-2	1024	4096	Long context	Low (API)

MiniLM v2

Use Case	Recommended Chunk Size	Overlap	Strategy
General Q&A over documents	400-600 tokens	50-100 tokens	Recursive Character
Code retrieval	Full functions/classes	0 (function boundary)	Language-aware splitter
Legal / contract analysis	300-500 tokens	100 tokens	Semantic or Structure-aware
Medical literature	200-400 tokens	50 tokens	Semantic Chunking
Customer support FAQ	100-200 tokens	20-50 tokens	Sentence-level
Long research papers	500-800 tokens	100-150 tokens	Semantic or Header-based
Real-time fact retrieval	50-150 tokens (propositions)	0	Proposition-based

MySQL → B-Tree

Vector Database

- Retrieval
- Indexing
- features

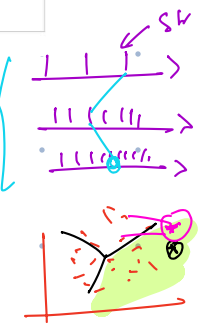
BM25
+ Dense

Database	Deployment	Scale	Hybrid Search	Metadata Filter	Best Use Case
(Chroma)	In-process or server	< 1M	No	Yes	Dev, small projects
Pinecone	Cloud managed	1B+	Yes (sparse)	Yes	Managed production
Weaviate	Self-hosted / cloud	100M+	Yes (built-in BM25)	Yes	Hybrid search prod
Qdrant	Self-hosted / cloud	100M+	Yes (sparse vectors)	Yes (rich)	High-perf production
(pgvector)	(PostgreSQL extension)	10M	With extensions	Full SQL	Postgres shops
Milvus / Zilliz	Distributed / cloud	Billions	Yes	Yes	Enterprise scale

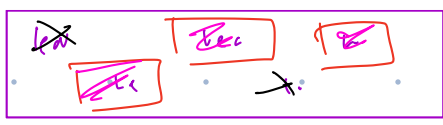
Hierarchical
Navigable
Small
Word

Inverted
file
index

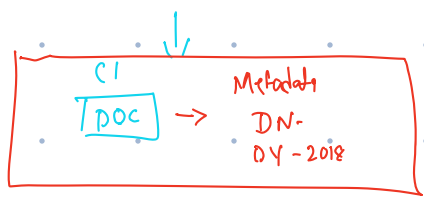
Index Type	Recall	Speed	Memory	Build Time	Best For
Flat (Exact)	100%	Slow (O(N))	Low	Instant	< 100K vectors
HNSW	95-99%	Very fast	High	Moderate	Most production RAG
IVF-Flat	90-98%	Fast	Low-Medium	Moderate	100K-10M vectors
IVF-PQ	85-95%	Very fast	Very low	Slow	10M+ vectors, memory constrained
ScaNN (Google)	98%	Fastest at scale	Medium	Slow	Extreme scale



Use Case	Recommended Chunk Size	Overlap	Strategy
General Q&A over documents	400-600 tokens	50-100 tokens	Recursive Character
Code retrieval	Full functions/classes	0 (function boundary)	Language-aware splitter
Legal / contract analysis	300-500 tokens	100 tokens	Semantic or Structure-aware
Medical literature	200-400 tokens	50 tokens	Semantic Chunking
Customer support FAQ	100-200 tokens	20-50 tokens	Sentence-level
Long research papers	500-800 tokens	100-150 tokens	Semantic or Header-based
Real-time fact retrieval	50-150 tokens (propositions)	0	Proposition-based

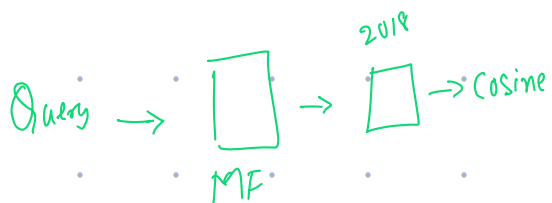


1000 Metafile > 300 → Dense
2018

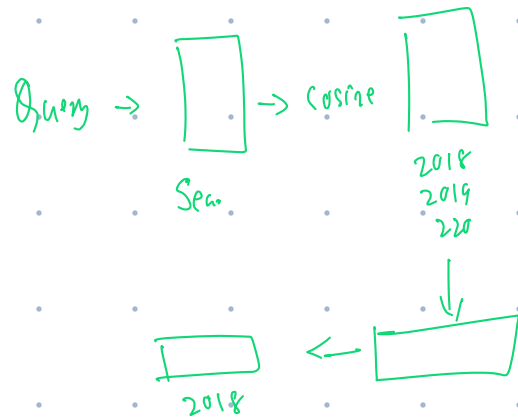


Two places

Before
Retrieval



After
Retrieval



Query - Embeddings → Text 3 small
↳ 1536

$$\begin{array}{r} \wedge \\ A \cdot B \\ \hline |A| |B| \end{array}$$

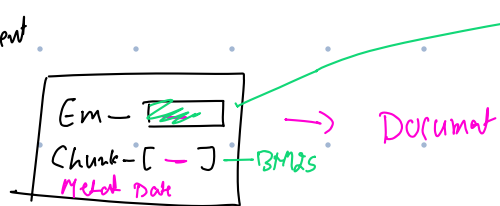
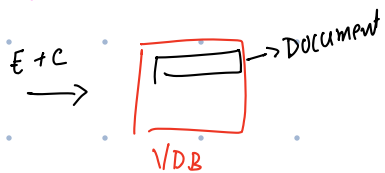
$$\frac{1536 \cdot 384}{}$$

Vectors - VDBs

Min 1642 → 384
↳ 10000
↳ 384

→ LLM

C1, C2, C3 →
e1 ↳ em
c2 c3



Qve

Embedding

C1, C2, C3 + Q → SP